Terrain Themer

Lawrence Winters, Natheniel Thoma, Matt Loots, CJ Osborne

1 Introduction

Procedural content generation (PCG) is a common technique used in modern game development. It allows for the creation of large and diverse environments without the requirement of handmade assets. One of the most widely used applications of PCG is terrain generation, where algorithms are used to generate landforms, biomes, and environmental features, creating the world the player experiences.

For our final project of this AI for Games course, we developed a terrain themer which is a system that procedurally generates terrain into meaningful visual biomes such as forests, deserts, and snow. This project was made in Unreal and uses noise generation algorithms, such as Perlin Curl and Voronoi, to simulate the generation of biomes and a rough landscape.

This paper explores the technical implementation of our project, as well as a brief discussion on the mathematical foundation for noise generation. We also examine real-world applications of similar systems and reflect on the development process.

2 Background

Procedural terrain generation has been used in games and simulations since the 1980s to create art assets and large environments with minimal manual work. Its popularity has risen over the past 40 years due to its ability to provide variation, scalability, and replayability in modern game design. Notable games such as *Minecraft* and *No Man's Sky*, shown in **Figure 1**, have utilized the power of procedural generation to create limitless worlds.

Figure 1 World generation in *Minecraft* (left) and *No Man's Sky* (right).

The foundation of procedural terrain generation comes from the use of mathematical noise functions. Perlin noise, developed by Ken Perlin in 1983, and its successors are commonly used to produce natural-looking visual effects. These algorithms generate smooth

and continuous noise by interpolating between random gradients at specific grid points. The resulting output can be used in a variety of ways and in our case, to generate terrain heightmaps and influence biome classification. Perlin Curl noise, a derivative used for vector fields, introduces patterns of turbulence that form flow fields, adding further realism to the terrain shape. A visual demonstration of both Perlin noise and a Perlin curl vector field are shown in **Figure 2.**

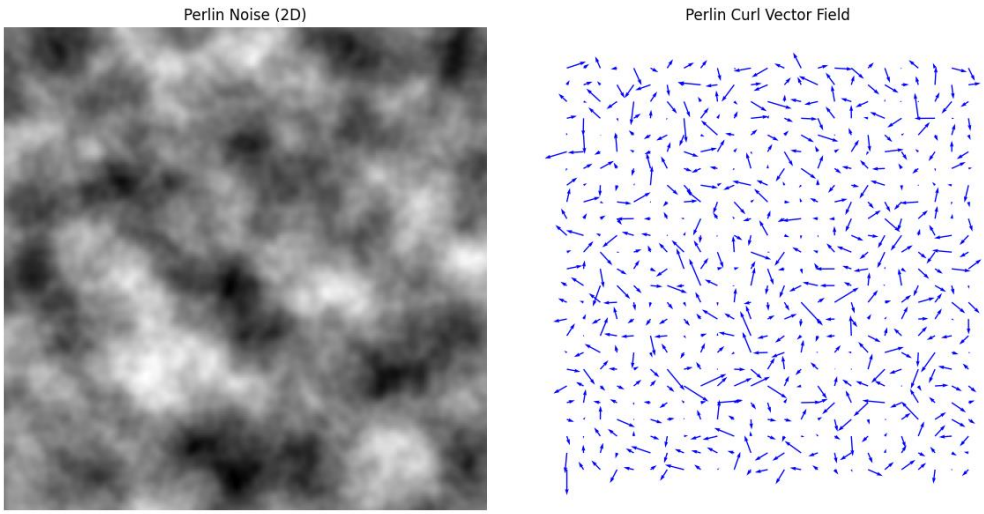

Figure 2 A visualization of Perlin noise and Perlin Curl vector field using Python.

In many systems, raw height data from noise functions is not enough to produce sufficient looking terrain for the players. To enhance the effects of the procedural terrain, developers often apply some sort of **theming system** that will classify terrain into biomes based on a variety of preset rules. These biome systems help create visually pleasing terrain and support gameplay design.

Our project builds on these established ideas by combining noise-based terrain generation and biome coloration in Unreal Engine. The goal was to use procedural algorithms to shape the terrain and biome textures through linear interpolation based on world position data and water height level.

3 Development & Implementation

Our terrain themer was developed entirely in Unreal Engine using the Blueprint visual scripting system. The project uses Unreal's built-in noise functions to procedurally generate terrain and apply visual theming based on Voronoi noise and world position data. Our project contains four main rendering modes: terrain texturing, height gradient, biome generation, and noise previewing.

In the terrain texturing mode, we blended between three terrain texture pairs based on elevation. These pairs are sand-to-grass, grass-to-rock, and rock-to-snow. Each pair is linearly interpolated using preset texture samples and the built-in lerp function, based on the water level in the world. This results in a visual transition between different terrain zones,

with textures adapting to the water's elevation changes.

The height gradient mode builds on the same structure but focuses on color visualization instead of textures. Each of the three terrain pairs, sand-to-grass, grass-to-rock, and rock-to-snow, are associated with a predefined color. These pairs are then blended based on gradients, and the results are combined to form a height-based terrain visualization. An example of the blueprint code for determining the combined color gradient is shown below in **Figure 3**.

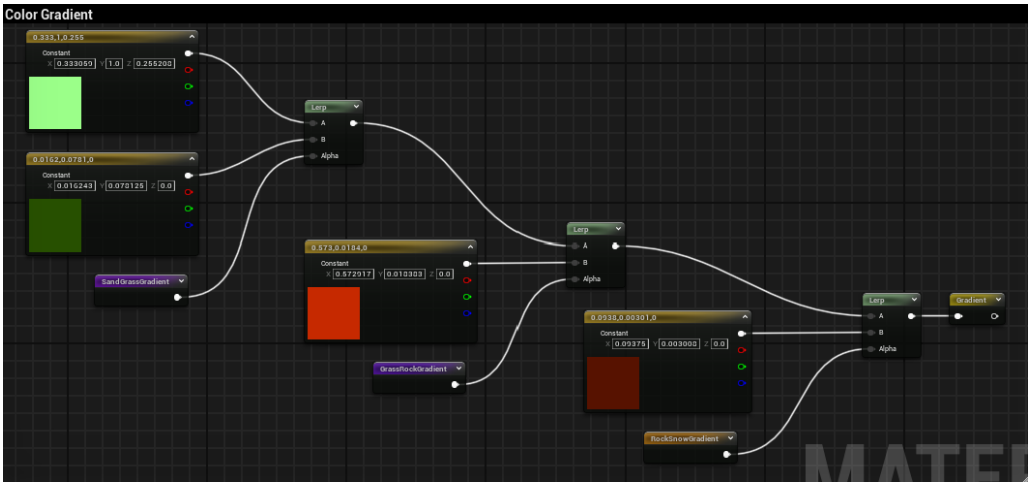

Figure 3 The color gradient blueprint used on the terrain material in Unreal.

The biome generation mode assigns colors to terrain based on Voronoi noise and world position data. The system uses a custom material function called BiomeNoise**,** which takes in the world position and a noise offset vector and passes it through the Voronoi noise function to create five distinct biome layers. These biome layers are separated by using five grayscale shades and then passed through a smooth step function, showcased in **Figure 4**, to produce soft transitions. Each of these five layers are combined with a texture that is assigned to a preset color. Tthe final output is a combined biome texture that visually distinguishes terrain regions by biome.

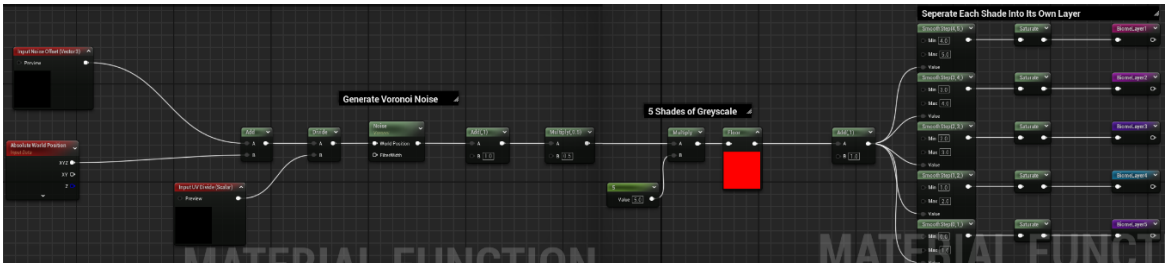

Figure 4 The color gradient blueprint used on the terrain material in Unreal.

Lastly, the show noise mode previews the raw output of various noise functions for debugging and visualization purposes. This is handled by a material function called HeightNoise, which supports five noise types: Simplex, Gradient, Value-Computational, Fast Gradient, and Perlin Curl. Each noise type can be visualized individually to compare

their characteristics and effects on terrain generation.

These four modes provided us with the necessary tools to test, visualize, and develop our terrain theming pipeline. Using Unreal's material system and visual scripting allowed for easy implementation, which was essential to the development process, because it allowed for minimal backend work. This gave us more time to focus on refining our terrain and biome generation

4 Noise Generation

At the heart of our procedural terrain system is the use of noise functions to simulate natural variation in elevation and biome distribution. Noise functions generate pseudo-random values that interpolate smoothly across the given area, attempting to mimic the organic patterns found in real-world terrain. We used a variety of noise types in Unreal Engine, allowing the user to test different height generation and biome classification.

4.1 Heightmap Noise

The terrain heightmaps were generated using noise functions accessed through a custom material function called HeightNoise. This function allowed us to preview and compare five different types of procedural noise:

Simplex Noise - an improvement over classic Perlin noise that has no noticeable directional artifacts, otherwise known as being visually isotropic.

Gradient Noise – also simply known as Perlin noise, creates pseudorandom gradients which are then interpolated between to produce smooth texturing.

Value-Computational Noise – a simpler form of noise based on interpolated random values from a created grid of points.

Fast Gradient Noise – a highly efficient method for generating noise patterns in materials. It uses a pre-computed 3D texture for rapid lookups, making it significantly faster than other noise functions that rely on more complex calculations or 2D texture lookups.

Perlin Curl Noise – showcased in **Figure 2**, uses similar math to Perlin noise but has the addition of a curl function which allows it to generate a turbulent noise. This resulting noise is incompressible, meaning that particles cannot converge to sink points where they get stuck.

Each of these individual noise types can be toggled in the display by clicking the toggle button located under the “Current Noise” text. Along with seeing the display of the output noise map, the user can examine how the differing noise functions influence the terrain generation. An image showcasing our project and the generated noise map can be seen in **Figure 5** below.

Figure 5 A display showcasing the noise mode, gradient, in our Unreal project.

4.1 Biome Noise

For biome classification, we used a material function named BiomeNoise. This function passes world position coordinates, and a noise offset vector through a Voronoi noise function. Voronoi noise divides space into cell blocks based on the proximity to randomly distributed points. This makes it suitable for distinguishing between different terrain regions like desert, forest, and snow.

The Voronoi noise output is divided into five biome regions using a SmoothStep function, which creates soft transitions and avoids harsh edges between biomes. Each of these five regions corresponds to a distinct input texture sample representing a unique biome. We apply the Voronoi masked output to these biome textures individually and then combine them, adjusting saturation. This results in a full-color biome map that visually distinguishes each terrain type.

Finally, this output biome noise map is linearly interpolated with a base UV texture using an alpha mask based on water level. This ensures that the parts of the map which are under the water level appear submerged. By combining scalar and vector noise, grayscale shading, and smooth interpolation, this system can produce terrain that is natural-looking and clearly separated.

An example of the resulting biome classification map is shown in **Figure 6**, showing how different biomes blend smoothly across the terrain while also handling the height-based water boundaries.

Figure 6 The full combination of the biome generation and noise mapping.

5 Real World Applications

Procedural terrain generation using noise functions and biome classification is applicable beyond our project. Within the games industry, this type of generation has enabled the creation of expansive environments with minimal human design. This is ideal for survival games like *Minecraft*, *No Man's Sky*, and *Valheim*. These games rely on massive open world exploration and are made replay able through the pseudorandom world generations. By combining noise-based heightmaps with biome logic, developers can simulate environments that are generated dynamically based on world coordinates, temperature gradients, or elevation.

When looking beyond the games industry, procedural terrain generation is used in fields such as virtual simulation, scientific research, and artificial intelligence. An example use of terrain generation is in autonomous vehicle simulations, which require large-scale, randomized terrains to train and test models safely. Similarly, architects can use these methods to see the environmental impact of buildings or plan developments based on differing terrain scenarios.

Our project illustrates how these systems can be prototyped within a commercial grade engine like Unreal, and how these tools are useful in a variety of fields.

6 Conclusion

This project explored the implementation of procedural terrain generation using a combination of noises such as Perlin Curl, Simplex and Voronoi within Unreal Engine. By integrating scalar and vector noise with biome classification through smooth interpolation, we were able to create a terrain system that is adaptable to any guide of generation. The layering of biome textures, combined with water level masking, resulted in terrain that supports variation and potential for environmental generation.

Through this process, we gained experience working with mathematical noise, material functions, and shader-based logic. The outcome demonstrates the power of procedural content generation and the accessibility of these techniques to designers and developers using industry-standard tools.

7 References

7.1 Online Documents:

[Adamic 99] Adamic, L.A., and B.A. Huberman. 1999. The nature of markets in the World Wide Web. Working paper, Xerox Palo Alto Research Center.
http://www.parc.xerox.com/istl/groups/
iea/www/webmarkets.html (accessed March 12, 2001).
[U.S. Census Bureau 00] U.S. Census Bureau. 2000. Health insurance coverage status and type of coverage by sex, race, and Hispanic origin. Health Insurance Historical Table 1.
http://www.census.gov/hhes/hlthins/historic/hihisttl.html.

https://www.cs.cmu.edu/~112/notes/student-tp-guides/Terrain.pdf

https://arxiv.org/html/2410.15644v1